EXPERIMENT-9

Name: Adavan R **no:240701623**

AIM:

The aim is to design input forms that validate data, such as email and phone number, and display error messages using **HTML/CSS and JavaScript with Validator.js.**

DESCRIPTION:

This experiment demonstrates how client-side validation works in web applications. It focuses on preventing invalid data entry by validating user inputs such as email and phone number.

An Event Registration Form is created for a college fest or seminar, where users enter their details like name, email, phone number, and select an event.

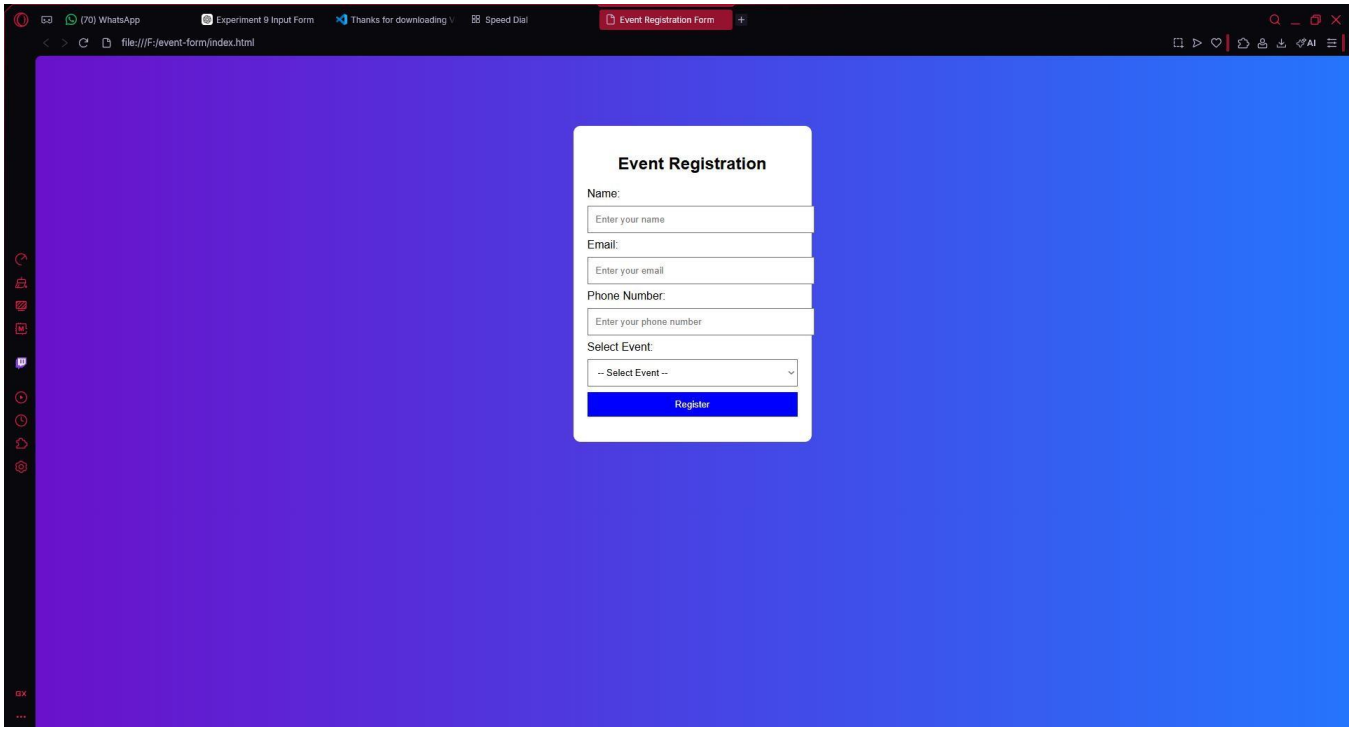
Validation ensures that:

- All required fields are filled
- Email is in correct format
- Phone number is valid

Step 1:

In this step, an HTML form is created to collect user data.

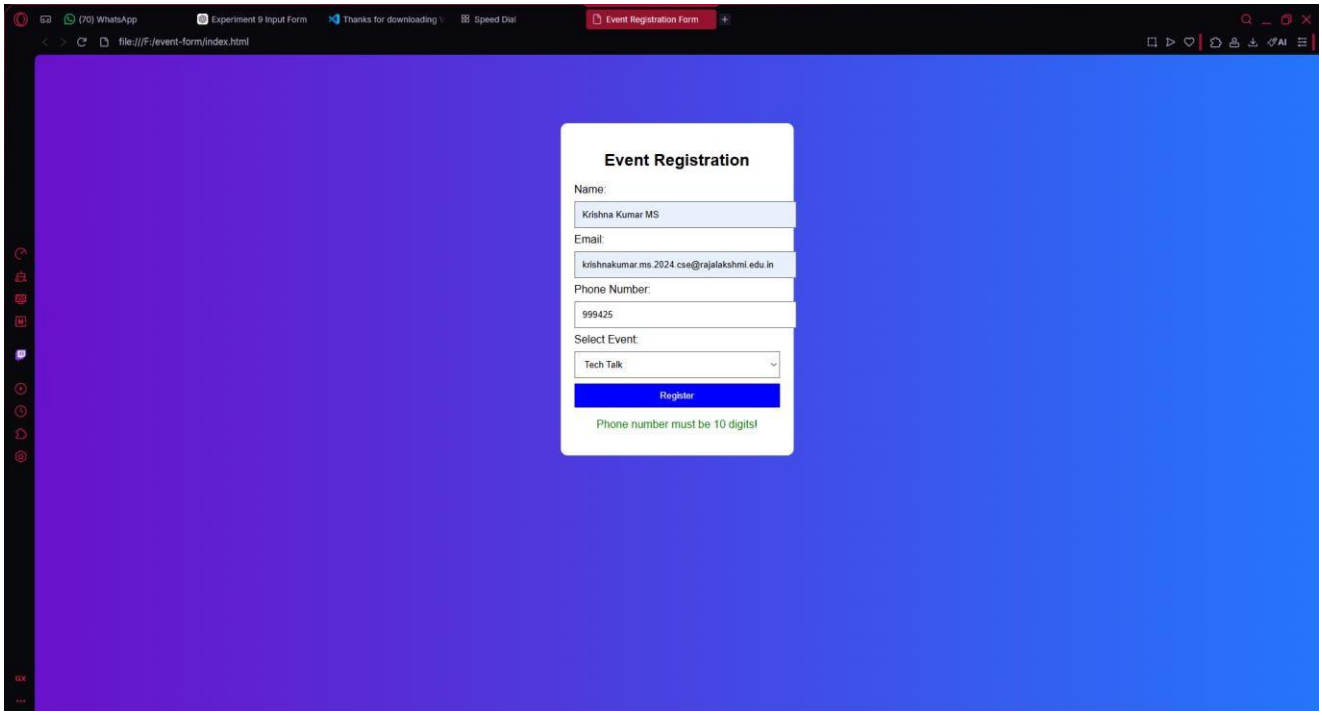
- A <form> element is used to group all input fields.
- Input fields are created for:
  - Name
  - Email
  - Phone Number
- A dropdown menu is added for event selection.
- Labels are used for clarity.
- A submit button is provided for form submission.



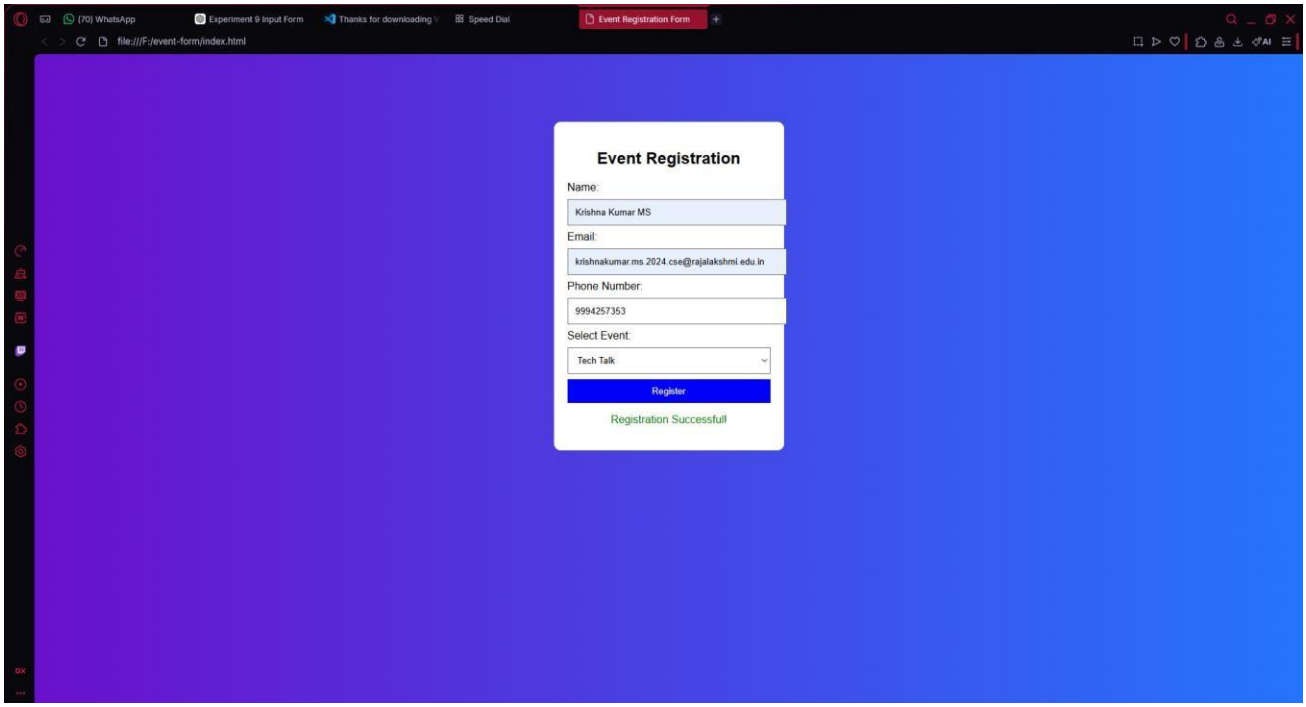
STEP-2:

In this step, CSS is applied to enhance the appearance of the form.

- Background is styled using gradient colors.
- Form container is centered on the screen.
- Input fields are styled with padding and spacing.
- Button is styled with colors and hover effects.



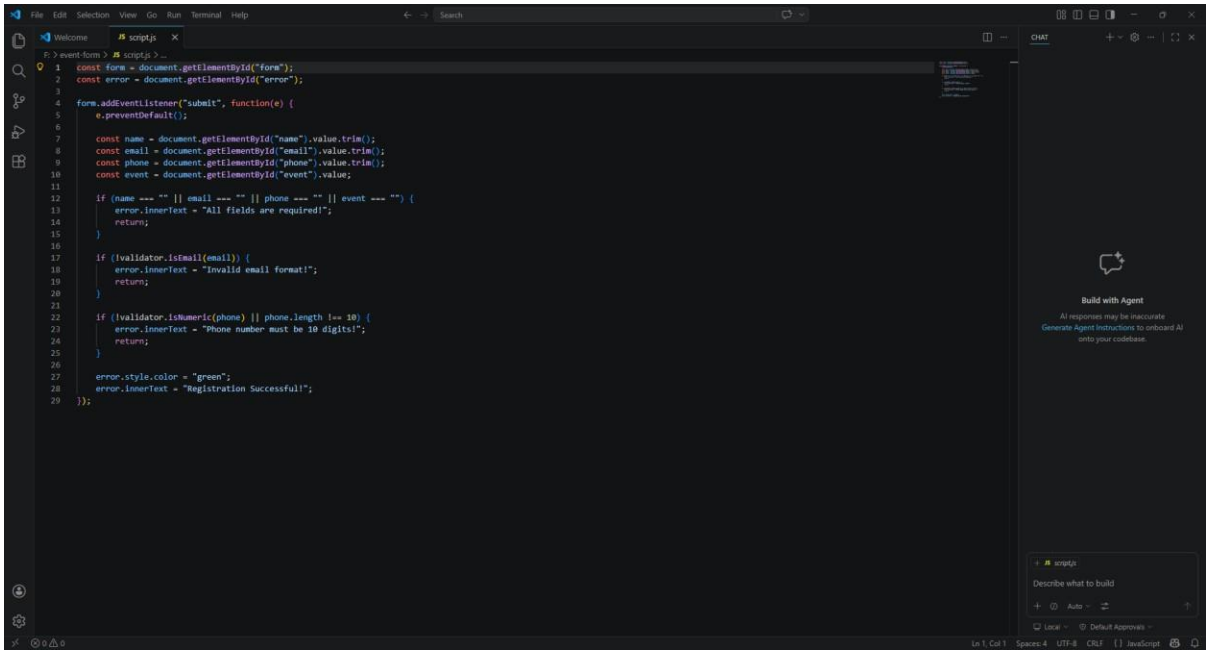
The input fields are styled to have proper width, spacing, and padding so that users can easily enter their data. The submit button is also designed with background color, text color, and hover effects to make it more noticeable and interactive.



This new design includes:

- Animated gradient background with floating elements
- Glassmorphism card design with blur effects
- Enhanced form fields with modern styling
- Smooth animations and micro-interactions
- Better typography and spacing
- Ripple effects on buttons
- Shake animations for errors

Step 3: Adding JavaScript for Validation



```javascript
const form = document.getElementById("form");
const error = document.getElementById("error");

form.addEventListener("submit", function(e) {
    e.preventDefault();

    const name = document.getElementById("name").value.trim();
    const email = document.getElementById("email").value.trim();
    const phone = document.getElementById("phone").value.trim();
    const event = document.getElementById("event").value;

    if (name === "" || email === "" || phone === "" || event === "") {
        error.innerText = "All fields are required!";
        return;
    }

    if (!validator.isEmail(email)) {
        error.innerText = "Invalid email format!";
        return;
    }

    if (!validator.isNumeric(phone) || phone.length !== 10) {
        error.innerText = "Phone number must be 10 digits!";
        return;
    }

    error.style.color = "green";
    error.innerText = "Registration Successful!";
});
```

In this step, JavaScript is used to validate user inputs.

- The Validator.js library is used to simplify validation.
- Email is checked using proper format validation.
- Phone number is checked to ensure it contains only digits and has 10 digits.
- If any field is empty, an error message is displayed.
- If all inputs are valid, a success message is shown.

Error messages are displayed dynamically to guide the user.

If any input is invalid, appropriate **error messages** are generated and displayed in the designated error message area.

This step ensures that incorrect or incomplete data is not submitted, thereby improving data accuracy, reliability, and user experience in the application.